ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 12

Topic: Static and Singleton

February 19, 2025

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 STATIC

The static keyword means that a member-like a field or method-belongs to the class itself, rather than to any specific instance of that class. As a result, we can access static members without the need to create an instance of an object.

1.1 The static Fields (Or Class Variables)

In Java, when we declare a field static, exactly a single copy of that field is created and shared among all instances of that class. It doesn't matter how many times we instantiate a class. There will always be only one copy of the static field belonging to it. The value of this static field is shared across all objects of the same class.

Imagine a class with several instance variables, where each new object created from this class has its own copy of these variables. However, if we want a variable to track the number of objects created, we use a static variable instead.

```
public class Car {
    private String name;
    private String engine;
    public static int numberOfCars;
    public Car(String name, String engine) {
        this.name = name;
        this.engine = engine;
        numberOfCars++;
    }
    // getters and setters
}
```

As a result, the static variable `numberOfCars` will be incremented each time we instantiate the Car class. Let's create two Car objects and expect the counter to have a value of two:

```
@Test
public void
   whenNumberOfCarObjectsInitialized_thenStaticCounterIncreases() {
    new Car("Jaguar", "V8");
    new Car("Bugatti", "W16");

    assertEquals(2, Car.numberOfCars);
}
```

As we can see, static fields can come in handy when:

- the value of the variable is independent of objects

- the value is supposed to be shared across all objects
- Lastly, it's important to know that static fields can be accessed through an instance (e.g. ford.`numberOfCars++`) or directly from the class (e.g., Car.`numberOfCars++`). The latter is preferred, as it clearly indicates that it's a class variable rather than an instance variable.

1.2 The static Methods (Or Class Methods)

Similar to static fields, static methods also belong to a class instead of an object. So, we can invoke them without instantiating the class. Generally, we use static methods to perform an operation that's not dependent upon instance creation.

For example, we can use a static method to share code across all instances of that class:

```
    static void setNumberOfCars(int numberOfCars) {
    Car.numberOfCars = numberOfCars;
}
```

Additionally, we can use static methods to create utility or helper classes. Some popular examples are the JDK's Collections or Math utility classes, Apache's StringUtils, and Spring Framework's CollectionUtils.

The same as for static fields, static methods can't be overridden. This is because static methods in Java are resolved at compile time, while method overriding is part of Runtime Polymorphism.

The following combinations of the instance, class methods, and variables are valid:

- instance methods can directly access both instance methods and instance variables
- instance methods can also access static variables and static methods directly
- Static methods can access all static variables and other static methods
- Static methods can't access instance variables and instance methods directly.

1.3 The Static Code Blocks

Generally, we'll initialize static variables directly during declaration. However, if the static variables require multi-statement logic during initialization we can use a static block instead.

For instance, let's initialize a List object with some predefined values using a static block of code:

```
    public class StaticBlockDemo {
    public static List<String> ranks = new LinkedList<>();
    static {
        ranks.add("Lieutenant");
        ranks.add("Captain");
        ranks.add("Major");
    }
    static {
        ranks.add("Colonel");
        ranks.add("General");
    }
}
```

As we can see, it wouldn't be possible to initialize a List object with all the initial values along with the declaration. So, this is why we've utilized the static block here.

A class can have multiple static members. The JVM will resolve the static fields and static blocks in the order of their declaration. The main reasons for using static blocks are:

- to initialize static variables, it needs some additional logic apart from the assignment
- to initialize static variables with a custom exception handling

1.4 Static Class

A static class is a class that cannot be instantiated, and all members of the class are static.

- No Instantiation: Cannot create an instance of a static class.
- Static Members: All methods and properties must be static.
- Memory Management: Static members are held in memory for the lifetime of the application domain.
- No Inheritance: Cannot inherit from or be inherited by other classes.

In Java, a class can be static if it is nested within another class.

1.5 Static Import

Static imports are initiated with the import static keyword. Their primary purpose is to streamline the process of accessing static members of a class. Static members, such as static variables and static methods, belong to the class itself rather than to instances of the class.

Without static imports, Java programmers would need to prefix the class name before accessing these members, adding verbosity to the code.

```
import static packageName.className.staticMember;
```

1.5.1 When to use static import

One common scenario where static imports are advantageous is when dealing with constants. By employing static imports, you can seamlessly use constants from other classes without needing to reference the class name. Consider the following example:

```
// Without static import
double circleArea = Math.PI * Math.pow(radius, 2);
// With static import
import static java.lang.Math.PI;
import static java.lang.Math.pow;
double circleArea = PI * pow(radius, 2);
```

1.5.2 When Not to Use Static Imports

While static imports offer convenience, excessive use can lead to ambiguity and confusion, particularly when multiple classes define the same-named static member. In such cases, it's best to avoid static imports to maintain code clarity.

```
// Multiple classes with static method 'print'
import static com.example.Printer.print;
import static com.anotherexample.Printer.print;

// Instead, qualify the usage
com.example.Printer.print("Hello");
com.anotherexample.Printer.print("World");
```

Oracle documentation about static import says developers to use static import very sparingly. Because it can make our program unreadable. Readers of the code can not understand which class that variable came from.

Why non-static variables cannot be referenced from a static context in Java?

2 SINGLETON

A singleton class in Java is a special class that allows only one instance (or object) of itself to be created. Imagine it like a unique key that opens a special door. No matter how many times you try to create a new object from a singleton class, you'll always get the same instance that was created initially. So whatever modifications we do to any variable inside the class through any instance, affects the variable of the single instance created and is visible if we access that variable through any variable of that class type defined.

2.1 Purpose of Singleton Class

- The primary purpose of a Java Singleton class is to restrict the limit of the number of object creations to only one. This often ensures that there is access control to resources, for example, a socket or database connection.
- Memory space wastage does not occur with the use of the singleton class because it restricts instance creation. As the object creation will take place only once instead of creating it each time a new request is made.
- We can use this single object repeatedly as per the requirements. This is the reason why multi-threaded and database applications mostly make use of the Singleton pattern in Java for caching, logging, thread pooling, configuration settings, and much more.

2.2 How to Design/Create a Singleton Class in Java?

To create a singleton class, we must follow the steps, given below:

- Ensure that only one instance of the class exists.
- Provide global access to that instance by
  - Declaring all constructors of the class to be private.
  - Providing a static method that returns a reference to the instance.
  - The instance is stored as a private static variable.

2.3 Example of Singleton class:

2.3.1 Eager Initialization:

The instance is created as soon as the class is loaded, ensuring that the instance is always available when needed.

```
public class EagerSingleton {
    private static final EagerSingleton instance = new EagerSingleton()
    ;
    private EagerSingleton() {
        // Private constructor to prevent external instantiation
    }
    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

2.3.2 Lazy Initialization:

The instance is created only when the getInstance() method is called for the first time. This can save resources if the instance is not always needed. However, it's important to note that this approach isn't thread-safe and might lead to issues in a multithreaded environment.

```
    public class LazySingleton {
    private static LazySingleton instance;
    private LazySingleton() {
        // Private constructor to prevent external instantiation
    }
    public static LazySingleton getInstance() {
        if (instance == null) {
            instance = new LazySingleton();
        }
        return instance;
    }
}
```

2.4 Differences between Static class and Singleton class

Instantiation

Singleton: Allows controlled, single instance creation.

Static Class: Cannot be instantiated at all.

State Management
Singleton: Maintains state between method calls through instance variables.
Static Class: Maintains state through static variables, which can lead to less controlled state management.
Inheritance and Polymorphism Singleton: Can implement interfaces and inherit from other classes.
Static Class: Cannot participate in inheritance or polymorphism.
Memory Management
Singleton: Instance is created on demand, potentially saving memory if never accessed.
Static Class: Static members are always loaded into memory, which can increase the application's memory footprint.

2.5 Choosing Between Singleton and Static

The decision between using a singleton or a static class depends on the specific requirements of your application.

- Use a Singleton when: You need to ensure a single instance of a class with controlled access. You require lazy initialization and possibly thread safety. Your class needs to maintain state or implement an interface.
- Use a Static Class when: You have a collection of utility or helper methods that do not maintain state. You need to define extension methods. You want to group related static methods and constants logically.

2.6 Scenarios when You should prefer Singleton Pattern over Static Methods

2.6.1 Managing Application Configuration

- Scenario: You have an application that needs to read configuration settings from a file or database and make them accessible throughout different modules.
- Advantage of Singleton: Implementing a Configuration Manager as a Singleton allows you to load configuration settings once and provide global access across the application. This ensures consistency in configuration retrieval and avoids redundancy that may occur with static methods, which cannot store state or adapt to changes in configuration sources dynamically.

2.6.2 Database Connection Pooling

- Scenario: Your application requires efficient management of database connections to handle multiple concurrent user requests.
- Advantage of Singleton: Using a Singleton pattern for a Connection Pool Manager ensures that there's a single instance managing the pool of database connections. This approach optimizes resource usage, facilitates connection reuse, and centralizes management of connection lifecycle events (e.g., initialization, validation, and cleanup) compared to static methods, which lack the ability to encapsulate connection state effectively.

2.6.3 Logging Service:

- Scenario: You need to implement a logging service that records events and errors from different parts of your application.
- Advantage of Singleton: Implementing the logging service as a Singleton allows all components and modules to log messages through a single, centralized instance. This ensures consistent formatting, filtering, and destination handling for log messages without the need to pass around a logger instance or rely on static methods that may not support dynamic configuration changes or thread safety as effectively.